

Deliverable 1

QA Readiness Guide

Instant USDC Reward Rail on Stellar - everything QA needs to execute the 28-case Deliverable 1 suite without touching code, secrets, or infrastructure.

This document answers issue **#13** (developer readiness). It is not an executed QA report and it records no verdict. QA execution, the revised test document, and the approval label all belong to issue **#80**.

Deliverable	D1 - Instant USDC Reward Rail on Stellar (Epic #4, issues #5-#13)
This document answers	Issue #13 - Prepare Deliverable 1 for unrestricted QA execution
QA executes against	Issue #80 - Deliverable 1 QA execution and test-case refinement
Test suite	Centient_D1_Test_Cases.docx - 28 cases, D1-TC-001 to D1-TC-028
Network	Stellar Testnet only. No mainnet transaction is authorised at any point in this suite.
Frozen build	fa36cf4663fecea6137dcd4e73b136baa49d34f0
Deployed at	2026-09-08T11:06:29.346Z
Environment	Railway project centient-work, environment named production, Stellar network testnet
Build URL	https://centient.work
Document date	2026-09-08
Owner	Centient engineering (developer side of the #13 / #80 split)
Approval signal	The QA:PASSED label on issue #80. Nothing else - no comment, no review link, no issue closure.

Read this first - what this document is and is not

It is the developer-owned half of the handoff: the frozen build, the live environment, the public on-chain evidence, the fixtures and access QA needs, and an honest per-case statement of what QA can execute today.

It is not a test result. Every case below is unexecuted. Status words in this document describe *developer readiness*, never a QA outcome.

It contains no secret material. No seed, private key, HMAC value, database URL, webhook URL, access token, password, signed pending XDR, or environment dump appears anywhere in it. Public Stellar addresses beginning **G** and testnet transaction hashes are public ledger data and are quoted deliberately.

Prepared by Centient engineering, 2026-09-08. Supersedes no earlier QA-facing document; the runbooks it cites remain the operational source of truth and are corrected where section 4 says so.

SECTION 1

Executive readiness assessment

The conclusion first, then the evidence for it.

Verdict

The QA test document is strong and needs only targeted tweaking. All 28 cases are well-formed, correctly severity-rated, and traceable to issues #5-#12. The revisions #80 needs are wording-level, not structural: replace the obsolete "#13 Final manual QA gate" framing and the "QA PASSED (full-sha)" verdict mechanic with the QA:PASSED label workflow, and replace the conditional phrases ("if supported", "where applicable", "controlled ability") with the concrete mechanisms named in sections 6 and 7 of this document.

Unrestricted QA is not ready to begin. Ten developer-owned items in the register below still block or degrade execution. Six are configuration or access gaps in the deployed environment; two are missing fault-injection mechanisms; one is an evidence gap in CI; one is a documentation correction shipped alongside this guide. None is a defect in the payout rail itself.

Recommended sequence. QA may begin the seven cases marked READY in section 6 immediately against the frozen build - they need only public Horizon data and the public health endpoint. Thirteen more are PARTIAL: their on-chain or automated half can be recorded now and the rest left open. The five BLOCKED cases should wait for the blockers that gate them, so that no case is executed twice or recorded against a half-configured environment.

Readiness by test case

Counts across all 28 cases in *Centient_D1_Test_Cases.docx*. The per-case detail is section 6.

Readiness	Cases	What it means for QA
READY	7	Executable today against the frozen build with only public data, the public wallet-health endpoint, or the repository. No developer action outstanding.
PARTIAL	13	Executable in part. The on-chain or automated half is available; the live-environment half needs a configuration item from the blocker register. Record what was observed and leave the rest open rather than marking the case Pass.
BLOCKED	5	Not executable by QA on this build. A developer-owned mechanism does not exist yet. Section 6 names the automated test that currently carries the invariant as the interim evidence path.
QA-OWNED	2	D1-TC-027 and D1-TC-028. These are #80's own evidence-package and verdict cases. They are rewritten by #80, not prepared by #13.

7 + 13 + 5 + 2 = 28. No case in the DOCX is unaccounted for.

Developer blocker register

Every item here is owned by engineering, not by QA. Each row states the exact exit condition, so #13 can be closed against evidence rather than against judgement. **B1 is resolved by the pull request that carries this document**; the rest are open.

ID	Sev	Developer-owned blocker	Cases held	Exit condition
B1	P0	Runbooks name a superseded payout account as current. GC5UOKLU . . . is documented throughout docs/ as the live payout account; the deployed rail actually uses GCP34RIT . . . QA following the runbook would verify the wrong account and could record a false pass on the strongest section of the suite.	TC-001, TC-002, TC-003, TC-004, TC-005, TC-020, TC-021, TC-027	Runbooks and the evidence index name the live account, with the earlier one clearly labelled historical. Shipped with this guide - verify at merge.
B2	P0	No scheduler, and no cron credential. CRON_SECRET is absent from the web service, so all four cron routes reject every request. Nothing in the repository schedules them either. Monitoring, retry-driven recovery, and reserve detection therefore never run on this build.	TC-024 (scheduler proof), TC-022, TC-023, TC-021; degrades TC-008 and TC-017 recovery	A cron credential is provisioned in its own secret store and an external schedule invokes POST /api/cron/wallet-health at least once per minute, with one authenticated response captured as evidence.
B3	P0	No alert channel. DISCORD_WEBHOOK_URL is absent, so every alert identity resolves to disabled. Alert delivery, cooldown, and de-duplication cannot be observed at all.	TC-022 (alert half), TC-024 (cooldown and de-duplication)	A dedicated QA webhook is configured on the web service and one alert is observed delivered, then suppressed inside its cooldown window.

ID	Sev	Developer-owned blocker	Cases held	Exit condition
B4	P0	No fault-injection mechanism for ambiguous submit or post-broadcast persistence failure. Neither the app nor the deployment exposes a test-only control that can produce an ambiguous Horizon response after submit while retaining the envelope identity, or fail local persistence after Stellar acceptance. QA cannot create these states by hand and must not be asked to patch code.	TC-010, TC-011	Either a testnet-gated, access-controlled fault-injection control exists and is documented, or #80 formally accepts the named automated tests in section 6 as the evidence path for these two cases.
B5	P1	Cold-reserve policy is not configured on the live web service. The cold account, its signer publics, and the trigger, target, and retained-minimum unit values are absent, so the reserve planner reports unconfigured and no refill can be detected or prepared from this build.	TC-021; the live half of TC-020	The five cold policy values are set on the web service and POST /api/cron/reserve-refill returns a real plan state (healthy, refill_required, or insufficient_reserve).
B6	P1	The co-signer's read-only database role is unproven. The cosigner service carries both a co-signer database URL and a generic one. Variable presence cannot show which role the process actually connects as, and a read-write connection would dissolve the one boundary this topology genuinely enforces.	TC-016	A redacted permission-probe transcript shows the required reads succeeding and every write to decision data failing with permission denied, plus the users read failing. The generic database URL is removed from the cosigner service.
B7	P1	The deployed remote co-signer path is not proven end to end. The #7 payout-service proof ran through the gated local co-signer, so it demonstrates two-signature settlement but not the deployed service. The live 1 USDC payout on this build does carry two independent signatures (ops and policy - see section 3), which is strong but does not by itself establish which process produced the policy signature.	TC-013, TC-014, TC-015; the deployed-path half of TC-004	One payout settles through the configured co-signer URL, with the co-signer's own request log entry recorded alongside the transaction hash.
B8	P1	QA has no admin access role. The admin status-health view and both admin health endpoints require the SUPER_ADMIN role, and the schema offers no read-only alternative. QA cannot see the balance and degraded-state views the cases ask for.	TC-022, TC-023 (admin-view half), TC-017 (cap observation)	A dedicated testnet QA admin account is issued and its credentials delivered out of band. Acceptable on testnet; a read-only role is the mainnet follow-up.
B9	P1	No deterministic fixtures or reset path. The six payout states and four recipient shapes the cases require have no documented seeding command, and there is no reset procedure that clears QA state without concealing an already-broadcast payout.	TC-008, TC-012, TC-014, TC-017, TC-019; setup for most of #7	A fixture and reset procedure is published as section 7 specifies, and one QA-run rehearsal produces every listed state.
B10	P1	No CI run exists for the deployed SHA. The CI workflow triggers on pull requests only, so it never runs against a merge commit. The green payments-lane job for this work is recorded on the pull request head f0b3dd0625de, whose merge produced the deployed fa36cf4663fe. The case as written asks for the run at the exact SHA, and that run does not exist.	TC-025, TC-026	Either a CI run is triggered against the deployed merge commit, or #80 accepts the pull-request head run plus the merge chain as equivalent and says so in the revised case.

Two things that are deliberately not on this register

Signing-key co-location on the web service is an accepted design position, not a blocker. Centient decided against separate infrastructure for the testnet MVP; the boundary that is enforced is the one ADR-0001 describes, and it protects against a compromised application rather than a compromised Railway account. QA should record the accepted posture and move on. See section 8.

The DOCX revisions are QA-owned. The obsolete "#13 Final manual QA gate" heading and the "QA PASSED (full-sha)" verdict field must be revised by #80, not by engineering. Section 12 states the workflow they are replaced with.

Separately, and not QA-blocking: a legacy PAYOUT_PRIVATE_KEY value remains set on the live web service. It is an EVM/Celo-era variable that no Stellar payout path reads - the only remaining reference in the repository is a test that deletes it - and its value was briefly exposed in operator terminal output during this readiness review because a variable filter was too broad. Recommendation: rotate it and delete the variable before handoff. No rotation has been performed; that needs explicit authorisation.

SECTION 2

Testnet environment and frozen build

One environment, one commit. Everything QA records must name both.

The frozen build

Full commit SHA	fa36cf4663fecea6137dcd4e73b136baa49d34f0
Short form	fa36cf4 - use the full SHA in every evidence record
Deployed at	2026-09-08T11:06:29.346Z
Deploy result	Web and cosigner deployments both succeeded at this commit
Base branch	develop
Commit link	https://github.com/webnxt-2030/Centient/commit/fa36cf4663fe

Build freeze rule

Engineering holds environment-affecting changes for the duration of the QA run. If the deployed SHA changes for any reason, engineering notifies #80 in a comment naming the old and new SHA, and every case already executed against the old SHA is reset to Not Tested unless #80 explicitly accepts it as unaffected.

A QA pass binds to the exact full commit SHA tested. A pass recorded against a short SHA, a branch name, or "latest" is not a pass.

Deployed topology

Component	What it is	Reachable by QA
web	The Next.js application: labeler and admin UI, the payout service, the payout worker, the reconciler, and every API route including the four cron routes.	Yes - https://centient.work and the Railway domain web-production-05914d.up.railway.app
cosigner	The independent policy co-signer. Holds the policy signing key and its own daily cap; answers signing requests over an HMAC-authenticated transport and re-derives every payout from its own read-only view of the ledger.	No. The service has no public domain by design and is reached only over same-project private networking. QA observes it through payout outcomes and through developer-captured logs, not directly.
Postgres	One database instance shared by web and cosigner, the latter through a separate role that may only read.	No. QA needs no database credential anywhere in this suite.

Environment configuration relevant to the suite

Values only - no secret is reproduced. Where a variable is marked **absent**, that absence is the blocker named in the last column.

Setting	Value on this build	Consequence for QA
Railway project	centient-work	Environment is named product ion but the Stellar network configuration is testnet . The name is an environment label, not a network. Do not read it as mainnet.
Stellar network	testnet	Horizon is the public testnet instance. Every hash in section 3 resolves on testnet explorers only.
Isolation level	same-workspace	The accepted testnet posture from ADR-0001. It is refused outright when the network is public, which is the mainnet gate. Section 8.
Application daily cap	2000000000 units = 200 USDC	Rolling trailing 24 hours, measured over payout jobs that carry a hash, an amount, and a broadcast timestamp.
Co-signer daily cap	2000000000 units = 200 USDC	Current UTC calendar day, from 00:00 UTC, measured over the co-signer's own read-only view. Same number, two independent settings, two different windows - see section 9.
Cron credential	absent	B2 . All four cron routes reject every request while it is unset. No scheduled monitoring, retry invocation, or reserve check runs.
Alert webhook	absent	B3 . Every alert identity resolves to disabled. Nothing is delivered.
Cold reserve policy	absent on web	B5 . The reserve planner is unconfigured on this build. Do not present reserve automation as live-ready.

What QA can reach, and how

Surface	Address	Auth	Use in the suite
Public wallet health	/api/health/wallet	None	Live USDC and XLM balances, spendable XLM after reserves, per-asset status, and the active warning thresholds. The primary evidence surface for TC-022 and TC-023.
Labeler application	https://centient.work	QA account	Withdrawal path that starts a real payout. Needed for the #7 cases once fixtures exist.
Admin status-health view	/admin/status-health	SUPER_ADMIN	Balance and degraded-state view. B8 - no QA account exists yet.
Admin ops view	/admin/ops	SUPER_ADMIN	Payout and cap observation. B8 .
Cron routes	/api/cron/wallet-health /api/cron/reserve-refill /api/cron/payout-retry /api/cron/payout-reconcile	Bearer, cron credential	B2 - unusable while the credential is absent. When provisioned, engineering runs these and hands QA the redacted responses; QA never holds the credential.
Co-signer	private networking only	HMAC	Not directly reachable. Negative cases in TC-015 are executed by engineering against the private endpoint, with redacted responses handed to QA.
CI	Actions - payments-lane job	Public repo	The evidence surface for TC-025 and TC-026. No run exists at the deployed SHA (B10) - use run 34217894219 on the pull-request head.

Live wallet-health response captured for this build

Read at 2026-09-08 from the public endpoint, which needs no authentication. QA should re-read it at execution time and record its own values; balances move.

Field	Value	Reading
account	GCP34RITQIVSLHS5 . . .	The live payout account. Same account as section 3.
USDC	19	Below the 50 USDC warning threshold, so the USDC status is warn . This is a funding state, not a defect.
XLM	approximately 9999.9999	Ample fee and reserve headroom.
spendable XLM	approximately 9997.4999	Net of ledger reserve, liabilities, and sponsored reserves. This is the number the XLM threshold is evaluated against, not the gross balance.
USDC status	warn	Distinct from the XLM status, which is what TC-022 asks the endpoint to demonstrate.
XLM status	healthy	No XLM alert active.
overall healthy	false	Correct: one asset in warn makes the aggregate unhealthy. QA should expect false here and not read it as an outage.
warn threshold	50 USDC	The configured USDC warning boundary on this build.

What to record from this endpoint

For TC-022 and TC-023 the endpoint is the primary evidence surface, so capture the whole response rather than a single field. Four things make a reading usable later:

- **The timestamp of the read and the SHA** the environment was running. A balance without both is not evidence.
- **Both per-asset statuses separately**, not just the aggregate. TC-022 asks whether low USDC and low XLM surface as *distinct* states, and the aggregate flag alone cannot answer that.
- **Spendable XLM, not gross XLM**. The XLM thresholds are evaluated against the spendable figure after ledger reserve, liabilities and sponsored reserves. Recording the gross balance would compare against the wrong number.
- **The monitoring status field**, which is separate from the balance statuses. It is how the endpoint reports that a source could not be read - the distinction TC-023 exists to check, and the reason a missing metric must never be recorded as healthy.

SECTION 3

Public payout, signer, reserve and transaction evidence

Everything in this section is public testnet ledger data, independently re-verifiable from Horizon without any Centient credential. Each row was read back from Horizon on 2026-09-08 while preparing this guide.

Live payout account

Account	GCP34RITQIVSLHS5T4XZRENIBUS3T7FHL3VSR24GK7HPMHGAACKWK4BU0
Thresholds	low = 2, medium = 2, high = 2
Master signer	GCP34RITQIVSLHS5T4XZRENIBUS3T7FHL3VSR24GK7HPMHGAACKWK4BU0 - weight 1
Ops signer	GAHZKJFAWX3HXAYQFAPCJ3Y2DFHYB0CSNVBJTZRDYJXN763KIR6HZ2U6 - weight 1
Policy signer	GCAUNAS2ZHBROHNKP32KWMEPJGMX5XJMSGNKXV3YVMRZ72SFBL3PKHIU - weight 1
Balances read	19.00000000 USDC and 9999.9999400 XLM
USDC issuer	GBBD47IF6LWK7P7MDEVSCWR7DPUWV3NY3DTQEVFL4NAT4AQH3ZLLFLA5

The two co-signer keys are not hyperlinked, and that is deliberate. A signer public key does not have to be a funded Stellar account, and these two are not: Horizon answers 404 for them, exactly as it does for any address that holds no ledger entry. They are still valid signers on the account above, which is where their weights are recorded. **QA should not record a 404 on a signer key as a defect.** The same is true of the two cold-reserve co-signers further down.

Every signer has weight 1 and the payment threshold is 2, so any two of the three keys can authorise a payment and **no single key can - the master included**. That is the property TC-001 and TC-002 exist to verify, and it is readable directly from the account page above.

Configuration transaction (set-options)

Hash	e966c0a5c27cbe0253f2812d158b38f8...
Result	successful, ledger 4565086, 2026-09-08T06:30:17Z
Operations	3 - add ops signer weight 1, add policy signer weight 1, then set master weight 1 and low/medium/high all to 2
Signatures	1 - and this is correct. The setup runs while the account is still single-key, so the master alone meets the pre-configuration threshold. Once 2/2/2 is installed, changing the signer set is itself a two-signature operation.
Fee charged	300 stroops, paid by the payout account

TC-003 asks for set-options evidence. The single signature on this transaction is the expected result, not a finding - QA should not record it as a failure of the two-signature rule.

Successful two-signature USDC payout on this build

Fee-bump hash	5083dd72a16bfa749c6b302c293e9319...
Inner payment hash	96eb8522c82eeb962e642f020cbb1d02...
Result	successful, ledger 4565647, 2026-09-08T07:17:02Z
Amount	1.0000000 USDC - exact to seven decimals
Source	GCP34RITQIVSLHS5T4XZRENIBUS3T7FHL3VSR24GK7HPMHGA AKWK4BUO
Destination	GD6ZXKCKGCIREWYMY65J324GMMTZYSGRBKDGO6XKXPTWU6L6WKEEXDLE
Signatures	2 on the inner payment and 2 on the fee bump. Both envelopes are sourced by the payout account, whose thresholds are 2/2/2, so both must carry two signatures.
Signing keys	the ops and policy signers, identified from the envelope signature hints. The master key did not sign this payout.
Fee	200 stroops, fee account is the payout account - Centient pays the network fee, not the recipient

Why the signing-key detail matters to QA

TC-004 asks for a real two-signature payout and TC-005 asks who pays the fee. Both are satisfied by the transaction above, and it is stronger than the case requires: the two signatures came from the two non-master signers, so this settlement did not depend on the master key at all.

It does **not** establish which *process* produced the policy signature - the deployed co-signer or a locally held key. That is blocker **B7**, and it is the reason TC-013 to TC-015 are not yet fully executable.

Cold reserve account

Account	GDPGRS4P6UZZK23CKKELGLJAYTCAWPV4C7TH6Q322SF735A5H6U5XK5G
Thresholds	low = 2, medium = 2, high = 2
Master signer	GDPGRS4P6UZZK23CKKELGLJAYTCAWPV4C7TH6Q322SF735A5H6U5XK5G - weight 1
Ops signer	GDERX2QG4LY4SK6VHBX5VRKE2PFDB25ZCD30LUM4RDVAY43EOXBRZ4BC - weight 1
Policy signer	GA0TECDRSB5HHAJAM00MUYTD0R5HTFNSSSETEDNWJDxDFRBZVQM00XD6V - weight 1
Balance read	6.9000000 USDC and 9999.9999500 XLM
USDC trustline	fddea73e981194f020bcf531f961...
Set-options	05bbe397033a7984700b8d844cf1...
Initial funding	452fd68061ecae052ebd681ee470...
Refill	3a5969cdac22dad6646630c90cdb...

The 2-of-3 topology TC-020 asks about is verifiable on-chain right now. The **policy values** TC-021 asks about are a different matter: the trigger of 10 USDC, target of 12 USDC, and retained cold minimum of 5 USDC are the values used historically, and they are **not configured on the live web service**. That is blocker **B5**. Do not present the refill automation as live-ready, and do not record TC-021 as a pass on the strength of the historical transactions alone.

SECTION 4

Live deployment facts against older repository evidence

The repository contains a complete, correct evidence trail for an earlier payout account. QA must not confuse it with the deployed one. This section is the reconciliation.

The one mismatch that would produce a false pass

The #5, #6 and #7 runbooks document GC5U0KLU... as the current payout account. The deployed rail uses GCP34RIT... Both accounts are correctly configured 2-of-3 multisigs on testnet, so a QA engineer following the runbook would verify a real, valid, correctly-configured account - **and it would be the wrong one**. Every threshold and signer check would pass while saying nothing about the build under test.

This is blocker **B1**, and it is corrected by the pull request that carries this document: the runbooks are updated to name the live account and to label the earlier one historical. Verify that correction is merged before starting section 6.

Side by side

Fact	Live on this build	Older repository evidence	How QA should treat it
Payout account	GCP34RITQIVSLHS5T4...	GC5U0KLU6J2ER0ZYYP...	Use the live account for every case. Cite the older one only as history.
Ops signer	GAHZKJFAWX3HXAYQFA...	GDNL20G7XGBHTPNW4W...	Signer sets are entirely different keys. Comparing across accounts proves nothing.
Policy signer	GCAUNAS2ZHBR0HNKP3...	GB6NBHA5ML3D0AQXBS...	As above.
Set-options tx	e966c0a5c27cbe...	8a16d6236cbb0a...	TC-003 must cite the live set-options transaction.
Two-signature payout	5083dd72a16bfa... (1 USDC)	be8a6a2363401f... (spike, 1 USDC)	Both are genuine two-signature settlements. TC-004 should cite the live one; the spike remains valid #6 evidence for the sponsorship mechanics.
Payout-service proof	not re-run on this account	9f6b8ae96a2d17... (0.1 USDC)	Important: that proof used the gated <i>local</i> co-signer, so it does not demonstrate the deployed remote co-signer path. Blocker B7.
Sponsored recipient	GD6ZXKCKGCIREWYMY6...	GA4XSHGIOVGWI6ZGDI...	The older recipient is still the best zero-XLM evidence for TC-006; its sponsorship transaction is 63dba3526178.... The live recipient is simply the destination of the live payout.
USDC issuer	GBBD47IF6LWK7P7MDE...	same issuer	Unchanged. The one fact that carries across cleanly.
Cold reserve	GDPGRS4P6UZZK23CKK... - live, 6.9 USDC	same account	The account and its 2/2/2 topology carry across. Its <i>policy values</i> do not - they are unset on web (B5).
Daily cap	200 USDC on both gates	100 USDC in runbook examples	The runbook figures are worked examples, not the deployed value. Record 200 USDC.
Reserve policy	unset on the web service	trigger 10 USDC, target 12 USDC, retain 5 USDC	Treat the documented figures as the historical policy, not as live configuration.
Co-signer topology	separate service in the same project	an earlier draft described a separate project	The amendment in ADR-0001 is authoritative. Section 8.

The older evidence is not wrong and should not be deleted. It records genuine testnet work: the #6 spike proved sponsored account and trustline creation with a recipient that held and spent zero XLM, and the #7 proof drove the real payout path end to end. The defect is only that nothing said it had been superseded. Once the runbooks are labelled, both sets of evidence are usable - for different claims.

SECTION 5

Assessment by issue: #5 to #13, and #80

Developer readiness per implementation issue, and what remains before the cases that cover it can be executed without improvisation.

Issue	Area	Ready?	State of the developer side	Outstanding
#5	Multisig payout account	READY	Fully verifiable from public Horizon data right now: 2/2/2 thresholds, three weight-1 signers, master below the payment threshold, and a verifiable set-options transaction. Only public keys appear in the repository and environment.	Nothing, once B1 is merged.
#6	Testnet payout spike	READY	Strong. Sponsored account and trustline creation, a zero-XLM recipient, Centient as fee source, and exact USDC deltas are all evidenced on testnet, and the live build has its own successful 1 USDC payout.	A fresh recipient per QA run is preferable to reusing the recorded one; section 7 gives the flow.
#7	Payout service	PARTIAL	The strongest implemented section. Exact integer-unit amount handling, one mutex owning the whole build-sign-co-sign-submit cycle, ambiguity resolved by envelope identity rather than rebuild, and a documented retryable/permanent classification for every failure mode.	B4 - QA cannot induce ambiguity or post-broadcast persistence failure. B9 - no fixtures. Amount, idempotency and concurrency remain executable.
#8	Policy co-signer	PARTIAL	Deployed as its own service with its own variables, its own deploy trigger, an HMAC-authenticated transport with a replay window, and independent ledger re-derivation. The application refuses to start if it can see both the co-signer URL and the policy secret.	B6 - the read-only role is unproven. B7 - the deployed path has no end-to-end payout. The service has no public domain, so negative cases run developer-side.
#9	Daily payout cap	PARTIAL	Two genuinely independent gates: the application's rolling 24-hour window and the co-signer's UTC-day window, configured as two separate values and checked by two processes. Both set to 200 USDC on this build. Bypassing one leaves the other refusing.	B3 - the breach alert cannot be observed. B9 - rolling usage cannot be seeded. Boundary and independence testing is otherwise available.
#10	Hot/cold reserve	PARTIAL	The cold account exists with 2/2/2 thresholds and three weight-1 signers, holds 6.9 USDC, and has a complete transaction trail for trustline, set-options, funding and refill. No deployed service holds a cold seed.	B5 - the policy values are unset on web, so trigger, target and retained minimum cannot be exercised live. On-chain topology is verifiable now.
#11	Wallet health	PARTIAL	The public health endpoint works and reports both assets, spendable XLM net of reserves, per-asset status, and the active thresholds. The monitor reports every source failure as degraded rather than as a false zero.	B2 - no scheduler. B3 - no alert channel. B8 - no admin view access. The endpoint half is executable; the alerting half is not.
#12	Payments-lane proof	PARTIAL	A named CI job runs the lane, and a two-halved guard asserts that no module outside a justified allowlist can submit to Horizon, build a USDC payment, or read the platform signing secret - verified by injection, not by inspection - paired with boundary tests that refuse anything short of two distinct verified signatures on both envelopes.	B10 - CI never runs on a merge commit, so no run exists at the deployed SHA. The lane itself is green on the pull-request head.
#13	This developer gate	PARTIAL	This document is the deliverable. Environment, build, evidence, comparison, per-case mapping, and the blocker register are all recorded.	The ten open blockers in section 1. #13 must not be closed until they are actually resolved.
#80	QA execution	QA-OWNED	Not engineering's to complete. The DOCX is a strong foundation and needs targeted wording revisions, listed in section 12.	Blocked on #13. Begins when the blockers close; approval is the QA:PASSED label on #80 alone.

SECTION 6

Mapping for D1-TC-001 to D1-TC-028

All 28 cases from Centient_D1_Test_Cases.docx, in DOCX order, with the developer-side answer to how each one is executed against the frozen build. Readiness words describe developer readiness, never a QA result.

Blocker references are to the register in section 1. **Where a case is BLOCKED**, the named automated test is the interim evidence path: it already holds the invariant the case exists to check, and #80 decides whether to accept it or leave the case open.

#5 - Multisig payout account

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-001	READY	Open the live payout account on a testnet explorer or read it from Horizon. Record the three thresholds and every signer with its weight, then compare against section 3: 2/2/2, three signers, all weight 1. Use the live account, not the runbook's historical one (B1).	Explorer or Horizon account page. Section 3 of this guide is the expected state.
D1-TC-002	READY	No transaction needs to be built. The property is arithmetic on data QA has already recorded in TC-001: the highest single weight is 1 and the payment threshold is 2, so no one key reaches it. If a submitted single-signature attempt is wanted as well, engineering runs it and hands over the rejection - QA is never given a seed.	The account's own thresholds and weights, plus the named boundary tests in payout-submitter and payout-envelope.
D1-TC-003	READY	Open the live set-options transaction. Confirm it is on testnet, carries the three expected operations, and produced the account state from TC-001. Expect exactly one signature - section 3 explains why that is correct and not a finding.	The set-options hash in section 3.

#6 - Testnet payout spike

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-004	PARTIAL	The live build already has a successful two-signature 1 USDC payout - record its hash, both signature counts, the exact amount, and the destination delta. To drive a <i>fresh</i> payout through the application, QA needs the fixtures from B9. The deployed-co-signer half is B7.	The payout and inner hashes in section 3; explorer balance history for the destination.
D1-TC-005	READY	Inspect the fee-bump envelope of that payout. Confirm the fee account is the Centient payout account and the fee charged is 200 stroops, and that the recipient's XLM balance is unchanged across the payment.	Fee account and fee charged on the fee-bump transaction.
D1-TC-006	READY	Use the recorded sponsored recipient: it was created and given its USDC trustline in one sponsored transaction, received USDC, and held and spent zero XLM throughout. Record before and after balances for both assets from explorer history. A fresh sponsored recipient can be created on request - section 7.	The sponsored-trustline transaction and the recipient account history in section 4.

#7 - Payout service

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-007	READY	Amounts travel as integer units end to end and are never parsed through a float; one USDC is 10,000,000 units. Verify the live payout settled as exactly 1.0000000 USDC, then read the named amount test, which pins an exact round trip at dust, ordinary, and ceiling values. Zero-value payouts are refused before an envelope is built.	The live payout amount, plus <code>lib/stellar/___tests___/payout-amount.test.ts</code> .
D1-TC-008	PARTIAL	Replay the same payout reference before and after confirmation and confirm one settlement only. Needs the validated and already-paid fixtures from B9. The co-signer independently refuses a reference that already carries a broadcast hash, which is the second line of defence.	Payout records and destination balance; <code>cosigner-service.test.ts</code> for the refusal.
D1-TC-009	PARTIAL	Use the fixed concurrency count of twelve simultaneous payouts - that is the count the existing regression asserts, and matching it keeps QA's result comparable. Pass calculation: twelve distinct sequence numbers, zero duplicate ledger rows, zero duplicate on-chain transfers. Needs B9 to seed twelve payable references.	<code>lib/___tests___/payout-concurrency-db.test.ts</code> - it fails with one distinct sequence when the mutex is removed.
D1-TC-010	BLOCKED	Not executable by QA on this build (B4). No control can produce an ambiguous Horizon response after submit while retaining the envelope identity. The implemented behaviour is exact and worth recording from the runbook: the envelope hash is computed before submission, Horizon is polled for that exact hash, and a rebuild is licensed by one thing only - Horizon reporting the transaction absent in a lookup made after it has ingested a ledger whose close time is strictly past the envelope's expiry. Anything less is non-retryable and is never refunded.	Interim: the ambiguity cases in <code>payout-submitter.test.ts</code> , plus the payout failure runbook.
D1-TC-011	BLOCKED	Not executable by QA on this build (B4). Local persistence cannot be failed after Stellar acceptance from outside the process. Note honestly that this is also the rail's known residual: if the process dies between submit and resolution the in-memory envelope identity is lost. Section 10.	Interim: the persistence and recovery cases in <code>payout-submitter.test.ts</code> and <code>lib/___tests___/reconciler.test.ts</code> .
D1-TC-012	PARTIAL	Attempt a payout to a recipient with no USDC trustline, and to an unfunded destination. Expect a permanent classification, the payout marked failed with the balance refunded, retry budget consumed, no requeue, and no funds moved. Needs the two recipient fixtures from B9.	Error response and the absence of any successful transaction on the destination; the failure-mode table in the payout failure runbook.

#8 - Policy co-signer

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-013	PARTIAL	Tamper the destination, then the amount, then the reference, between validation and policy signing, and confirm the co-signer refuses each and issues no signature. The co-signer has no public domain , so engineering executes these against the private endpoint and hands QA the redacted refusals (B7). The co-signer returns a detached signature, never a transaction, and it is verified against our own envelope hash - so the worst a compromised co-signer can do is refuse.	Redacted co-signer responses; <code>cosigner-service.test.ts</code> and <code>payout-cosigner.test.ts</code> .
D1-TC-014	PARTIAL	Request a co-signature for a non-validated payout and for one already carrying a broadcast hash. Both must be refused with no signature and no new settlement. Needs B9 fixtures and, for direct observation, B7.	Redacted refusals plus the payout row and chain state.
D1-TC-015	PARTIAL	Four negative requests: no authentication, a stale timestamp outside the replay window, a replayed nonce, and a malformed body. All must fail closed with no signature. An unauthenticated request is refused before any database work. Executed developer-side (B7); QA receives status codes and redacted bodies. No HMAC value is ever shared.	Redacted HTTP responses; <code>cosigner-transport.test.ts</code> .
D1-TC-016	BLOCKED	Not executable by QA on this build (B6). QA holds no database credential and must not. The evidence has to be a developer-run, redacted permission probe: the required reads succeed, every write to decision data fails with permission denied, and the user-table read also fails. Variable presence alone proves nothing, and the cosigner service currently carries a generic database URL as well as the read-only one.	Interim: the probe transcript in the co-signer deployment runbook, once run against the live role; plus <code>cosigner-ledger-db.test.ts</code> .

#9 - Daily payout cap

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-017	PARTIAL	Both caps are 200 USDC on this build. Test just below, exactly at, and just above, at both decision points. Note the deliberate window difference - the application counts a rolling trailing 24 hours, the co-signer counts the current UTC day - so either may refuse what the other allows. That is fail-closed, not a defect. Needs B9 to seed rolling usage, and B8 to observe cap state in the admin view.	Cap arithmetic, both refusals, and the presence or absence of a settlement; <code>lib/__tests__/payout-cap.test.ts</code> .
D1-TC-018	PARTIAL	Both directions are mandatory. Direction one: set the application cap to 0, which disables only that gate, and confirm the co-signer still refuses. Direction two: raise the co-signer cap well above the request and confirm the application still refuses. Note that a co-signer cap of 0 is invalid - the service refuses to start - so direction two is done by raising it, never by zeroing it. Both are variable changes on separate services followed by a redeploy; no code change.	The refusal at the remaining gate and no on-chain payment; <code>cosigner-service.test.ts</code> runs both directions.
D1-TC-019	PARTIAL	Drive concurrent decisions near the cap and record what happens. Expect overshoot. The cap is a check, not a reservation, so concurrent payouts can each pass it and together exceed it. That is documented, asserted behaviour - record it as observed and do not claim a stronger guarantee. An unsafe <i>duplicate settlement</i> , by contrast, would be a blocking defect. Needs B9.	Concurrency results and the residual-risk note in section 10; <code>payout-concurrency-db.test.ts</code> asserts the overshoot deliberately.

#10 - Hot/cold reserve

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-020	PARTIAL	The on-chain half is executable now: read the cold account's 2/2/2 thresholds and three weight-1 signers from section 3. The single-signer refill rejection is a custodial ceremony run by engineering - the refill command refuses fewer than two valid signatures and refuses the same key twice - and QA receives the redacted refusal. QA is never given a cold seed, and no signed pending XDR is shared.	Cold account page; redacted refusal output; <code>reserve-refill.test.ts</code> .
D1-TC-021	BLOCKED	Not executable live on this build (B5). The trigger, target and retained-minimum values are absent from the web service, so no refill can be detected or prepared. The historical policy was a trigger of 10 USDC, a target of 12 USDC, and a retained cold minimum of 5 USDC, and the recorded refill transaction is in section 3 - but those are history, not live configuration. Worst-case hot-wallet loss is bounded by the target, because every refill restores exactly the target and nothing else deposits into the hot wallet.	Interim: the recorded refill transaction plus <code>reserve-refill.test.ts</code> and <code>cold-reserve-setup.test.ts</code> .

#11 - Wallet health

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-022	PARTIAL	The state half is executable now against the public endpoint, and the build is conveniently already in a mixed state: USDC is in warn at 19 USDC against a 50 USDC threshold while XLM is healthy, and overall healthy is false. That is exactly the distinct-per-asset behaviour the case asks for. The alert half is B3, and the admin-view half is B8.	The public health response; the alert identity list in the cold-reserve runbook; <code>lib/__tests__/health-monitor.test.ts</code> .
D1-TC-023	PARTIAL	The invariant is that a missing metric is never read as healthy - unavailable sources raise their own paging identities and balances report as null rather than zero. Inducing a Horizon or asset-source outage against the live build needs a developer-run control (B2 for the authenticated invocation). QA can record that the endpoint reports a monitoring status field distinct from the balance statuses.	The monitoring-status field in the health response; the monitoring-source identity table; <code>health-monitor.test.ts</code> .
D1-TC-024	BLOCKED	Not executable on this build (B2 and B3). The case requires proof that a scheduler actually invokes monitoring rather than a manual endpoint call, and that duplicate alerts are suppressed inside a cooldown. Neither the cron credential nor the alert channel is configured, and nothing in the repository schedules the route. When provisioned, note that the default cooldown is 15 minutes and QA should be given a shortened QA-only value rather than waiting it out.	Interim: <code>lib/__tests__/health-alert.test.ts</code> for cooldown and de-duplication. The scheduler half has no substitute - it is infrastructure.

#12 - Payment-lane proof

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-025	PARTIAL	Read the blocker first (B10): CI triggers on pull requests only, so there is no run at the deployed merge commit. The evidence that exists is CI run 34217894219 on f0b3dd0625de, the head of pull request #79, whose merge produced fa36cf4663fe. Both its jobs - payments-lane and build - are green. Record the chain explicitly rather than implying the run is at the deployed SHA. Then confirm the named payments-lane job ran and is green, that it contains the no-single-key guard, and that the typecheck, migration and build checks required by D1 are also green. The guard has two halves and both must be present: a structural scan of lib, app, services and scripts against a justified allowlist, and boundary tests refusing anything short of two distinct verified signatures on <i>both</i> envelopes.	CI run 34217894219 on the pull-request head; lib/s tellar/___tests___/no-single-key-payout.test.ts.
D1-TC-026	READY	Confirm the cap-alert ordering regression is present and green. The invariant: the alert is raised only after the broadcast tuple - hash, amount, broadcast time - has been persisted, at each of the three sites that persist it. The test controls read and write ordering rather than measuring timing, and it was confirmed to fail against the preceding commit.	lib/___tests___/payout-cap-alert-ordering-db.test.ts in CI run 34217894219 (B10 applies here too).

#13 and the QA record - now owned by #80

Case	Ready?	How QA executes it on this build	Evidence source
D1-TC-027	QA-OWNED	Keep the case, revise its wording. The evidence-package check is valid and useful, but it belongs to #80 and it must be re-scoped: drop the requirement that the package be produced by a #13 manual QA gate. Section 11 of this guide is the redaction standard to review against, and section 13 is the link index.	This guide, the runbooks it cites, and CI run 34217894219 - noting B10, that the run is on the pull-request head rather than the deployed merge commit.
D1-TC-028	QA-OWNED	Rewrite the verdict mechanic. Delete "Record QA PASSED (full-sha)" and replace it with: apply the QA:PASSED label to issue #80 when every pass condition there is met. Still record the full SHA, the environment URL, tester, date, and the executed and blocked case IDs in the document - a blocked case is not a pass. No verdict comment, Claude review link, issue closure, or Epic #4 update is required of QA.	The QA:PASSED label on #80. Section 12 states the whole workflow.

SECTION 7

Developer-owned fixtures, reset, access and fault injection

The contract behind #13: QA must be able to reach every precondition in the suite without a code change, a database credential, a signer seed, or an undocumented developer intervention. This section states what is provided, and states plainly where it is not yet.

The access rule, stated once

No QA step in this suite requires a signer seed, a database superuser role, a production credential, an HMAC value, or mainnet access. Where a case genuinely needs one of those - the single-signer refusals, the co-signer negative requests, the read-only role probe, the authenticated cron calls - engineering executes the step and hands QA a redacted result. Those steps are marked in section 6.

If QA is ever asked for a seed or a database URL to complete a case, that request is itself a defect. Stop and raise it on #80.

Fixtures QA needs (blocker B9)

The suite's preconditions reduce to two fixture families. Neither has a seeding command today; this table is the specification engineering owes #80, and the rehearsal that closes B9 is one QA-run pass that produces every row.

Fixture	Needed by	What it must look like
Validated payout	TC-008, TC-013, TC-017	A payout reference whose row passes validation, has a destination holding a USDC trustline, and has no broadcast hash. The signable baseline.
Non-validated payout	TC-014	Same shape, validation state not met. The co-signer must refuse it on its own ledger read, not on anything the request claims.
Already-paid payout	TC-008, TC-014	A reference that already carries a broadcast hash. The second co-signature request must be refused rather than producing a duplicate settlement.
Pending payout	TC-017	A payout deferred by a cap refusal: still pending, retry budget untouched, campaign debit still reserved.
Failed payout	TC-012	A payout that terminated on a permanent rail error, with the balance refunded and the retry budget consumed.
Reconciliation-required payout	TC-010, TC-011	A payout marked failed with a needs-manual-reconciliation reason and no refund. Cannot be produced without B4.
Recipient with trustline	TC-004, TC-008, TC-009	A funded testnet account holding a USDC trustline for the configured issuer. The ordinary success path.
Recipient without trustline	TC-012	A funded account with no USDC trustline. Expect a permanent no-trust classification and no funds moved.
Zero-XLM sponsored recipient	TC-006	An account created and given its trustline in one sponsored transaction, holding no XLM. The recorded example is GA4XSHGIOVGWI6ZG..., sponsored by transaction 63dba3526178...
Invalid destination	TC-012	An address that was never created on-chain. Expect a permanent no-destination classification.

Reset rules

A reset that hides a broadcast payout is worse than no reset, because it destroys the traceability the whole suite depends on. Three rules constrain whatever mechanism engineering ships:

- **Never delete or rewrite a payout row that carries a transaction hash.** It settled on a public ledger; the local row is the only record of why. Close it out, annotate it, and create a new reference instead.
- **Never reuse a payout reference across QA runs.** Idempotency is one of the properties under test; a recycled reference makes a genuine duplicate indistinguishable from correct replay refusal.
- **Record the reset in the evidence trail.** Each QA run notes what was reset, when, and the SHA - so a later reader can tell a deliberate reset from a disappeared payout.

Fault injection: what exists, and what does not (blocker B4)

Scenario	Case	Available to QA?	Interim evidence path
Ambiguous Horizon response after submit, retaining envelope identity	TC-010	BLOCKED	The ambiguity cases in the payout-submitter suite, plus the reconciliation procedure in the payout failure runbook.
Local persistence failure after Stellar acceptance	TC-011	BLOCKED	The persistence and recovery cases in the payout-submitter and reconciler suites.
Health source unavailable	TC-023	PARTIAL	Developer-run outage against the live build once B2 lands; the monitor's own degraded identities are asserted in the health-monitor suite.
Cap gate bypass, either direction	TC-018	PARTIAL	A variable change per service plus a redeploy. No code change. Note a co-signer cap of 0 is invalid - raise it instead.
Concurrency at a fixed count	TC-009, TC-019	PARTIAL	Twelve simultaneous payouts, matching the existing regression, once B9 provides twelve payable references.
Permanent rail errors	TC-012	PARTIAL	Two recipient fixtures from B9. No injection needed - the errors are real Horizon results.
Unauthenticated, stale, replayed, malformed co-signer requests	TC-015	PARTIAL	Developer-executed against the private endpoint; QA receives status codes and redacted bodies.

Any fault-injection control engineering does ship must be test-only, testnet-only, access-controlled, and unavailable in the public-network configuration - the same fail-closed shape as the existing gated local co-signer. A control that could be reached on mainnet would be a worse defect than the gap it closes.

SECTION 8

Co-signer topology and credential separation

What the deployed boundary actually is, stated at the strength it can honestly be claimed - which is narrower than the original design intent and wider than nothing.

Deployed topology

Shape	A separate cosigner service inside the existing centient-work Railway project. Not a separate project, and not a separate account.
Why	Project creation is unavailable to the maintainer's account in the workspace where Centient runs, so the separate-project topology in ADR-0001 could not be provisioned. The ADR was amended the same day it was accepted to record this.
Isolation level	The configured value is same-workspace, and it is refused outright when the Stellar network is public . That refusal is the mainnet gate.
Transport	HMAC-SHA256 over the raw request body, bound to a timestamp and a nonce with a replay window, compared in constant time.
Networking	Same-project private networking. The co-signer has no public domain, so neither the signing request nor the read-only database credential leaves Railway's network.
Replicas	Exactly one . The replay-nonce store and the record of signatures issued but not yet seen broadcast are both in-process; a second replica would silently stop guarding either.

What the boundary buys, and what it does not

Boundary	In force?	What it means
Separate process and container	Yes	The co-signer runs its own container from its own start command.
Separate variable scope	Yes	The policy signing key exists in exactly one service, and the application refuses to start if it can see both the co-signer URL and the policy secret. That refusal is what makes the boundary testable rather than aspirational.
Separate deploy trigger	Yes	Watch paths scope the co-signer's redeploy to its own source paths.
Separate database credential	Claimed, unproven	A role granted select on only the two tables the policy decision reads. Blocker B6 - the live role has not been probed, and the service also carries a generic database URL.
Independent re-derivation	Yes	The co-signer reads the row the reference names and independently confirms validation state, destination, and exact amount. It never trusts the presented values.
Independent cap gate	Yes	Its own daily cap value over its own read-only view. A cap of zero is refused at boot rather than treated as unlimited.
Separate member list	No	The specific cost of the same-project amendment. Anyone with access to the project can read both services' variables.
Separate account and control plane	No	One Railway account, one repository, one CI system, one database instance. This is the accepted residual, and it is why the level is refused on mainnet.

The honest claim, for QA's record

The boundary protects against a compromised application. It does not protect against a compromised Railway account or collaborator. An attacker holding the application container still cannot forge a payout - the detached co-signature is verified against our own envelope hash before it is merged, so the worst a compromised co-signer can do is refuse to sign. An attacker holding the Railway account reaches both signers.

In the SOW's terms this satisfies *server* isolation and not *account* isolation. Recording it at that strength - and not at the strength the original acceptance criteria implied - is a requirement of the QA record, not a caveat appended to it.

Key co-location on the web service is an accepted decision, not a finding. Centient chose against separate infrastructure for the testnet MVP while the value at risk is zero. QA records the posture in force at this SHA and does not raise it as a defect. It becomes a blocker at mainnet cutover.

Credential-separation checks QA can rely on

- **Only public keys appear in the repository and in the application environment.** Signer secrets are never committed. The payout account's policy signer public key is GCAUNAS2ZHBROHNKP32KWMEP....
- **The application cannot hold both signatures.** Seeing the co-signer URL and the policy secret together is a boot-time refusal, not a warning.
- **There is no single-signature fallback anywhere.** With no co-signer configured, the payout throws before building anything rather than degrading to one signature.
- **The gated local co-signer is not a deployment option.** It is refused on the public network, requires an explicit opt-in flag, and requires a policy secret that agrees with the configured public key. It exists for local proofs - and it is what the older #7 proof used, which is why that proof does not close B7.
- **Rotating the shared secret is a two-service change** with a window in which they disagree. Payouts fail closed with a 401 during it rather than proceeding unsigned. Rotating the policy key is not a variable change at all - it needs a new signer set on the payout account.

SECTION 9

Daily cap, reserve, wallet health, alerts and monitoring

The configured facts behind cases TC-017 to TC-024, separated from the parts that are not running on this build.

The two cap gates

	Application (web)	Policy co-signer
Configured value	2000000000 units = 200 USDC	2000000000 units = 200 USDC
Window	trailing / rolling 24 hours	current UTC calendar day , since 00:00 UTC
Spend source	payout jobs carrying a hash, an amount and a broadcast timestamp inside the window	its own read-only ledger view, plus signatures issued but not yet visible as broadcasts
Disabled value	0 disables this gate only - use it solely for an explicit test decision	none . Zero is invalid and the service refuses to start
On breach	throws a cap error before resolving the co-signer or building anything, then runs the cap-alert path	returns HTTP 409 and no detached signature, so the payout cannot be submitted whatever the application decided
Changed by	a variable on the web service, then a redeploy	a separate variable on the cosigner service, then a redeploy

Three cap facts that read as bugs and are not

The windows differ on purpose. At 00:30 UTC the co-signer counts only payouts since midnight while the application still counts the preceding 23.5 hours. Either may refuse what the other allows. A payout settles only when both windows have room - that is fail-closed, and unifying the windows would be a design change, not an operational fix.

The two values must be written twice. The co-signer deliberately does not fall back to the application's cap and the application does not read the co-signer's. Using the same number is fine; sharing one setting is not.

A cap alert reports the ledger's spend, not the refused amount. An alert can read below 100 percent consumed and still carry a blocked-attempt line. The units in that line never left the payout account. It is the refusal, not an arithmetic error.

Cap-breach behaviour by job type

TC-017 asks whether a refused payout stays transient without consuming its retry budget. The answer differs by path, and QA should record which path it exercised:

- **A withdrawal - the live earnings path - is terminal.** Its retry budget is consumed and the user's locked balance is refunded, so the money returns to their withdrawable balance and they can re-withdraw once the window has room. Nothing retries it on their behalf.
- **A legacy per-submission payout is deferred.** The submission stays pending, its retry budget is untouched, and its campaign debit stays reserved. This path drains only jobs enqueued before the accrual cutover.
- **The deferral is not self-healing.** Recovery runs through the payout-retry cron route, which is scheduled outside the application - and is **not scheduled on this build (B2)**. A deferred submission never retries while that schedule is absent, so do not read a pending payout as merely waiting.

Reserve policy

Cold account	GDPGRS4P6UZZK23CKKELGLJAYTCAWPV4C7TH6Q322SF735A5H6U5XK5G
Authorisation	2 of 3, thresholds 2/2/2, three weight-1 identities: cold master, cold ops, cold policy
Seeds held by services	none . The application stores only the cold public key, the two signer publics, and the policy unit values. Each custodian signs on isolated infrastructure with exactly one seed, and signing contacts no network.
Historical policy	trigger 10 USDC , target 12 USDC , retained cold minimum 5 USDC
Live configuration	absent on the web service (B5) . The reserve planner reports unconfigured, so no refill can be detected or prepared from this build.
Worst-case hot loss	bounded by the target , because every refill restores exactly the target and nothing else deposits into the hot wallet. The cold reserve is not at risk.
Submit-time checks	The submit step reloads both balances and re-checks that the refill neither raises the hot float above target nor pulls cold below its floor, and refuses extra operations, a different source or destination, a wrong asset or issuer, a fee above 10,000 stroops, missing or expired time bounds, duplicate signer identities, or fewer than two valid signatures.

A refill XDR expires 15 minutes after preparation. It carries no secret, but an unsubmitted signed XDR is approval material: **it is never committed, posted, or shared with QA**, and it does not appear in this document.

Wallet health and alert thresholds

Signal	Configured on this build	Note
USDC warn	50 USDC	Currently crossed - the build reports USDC status warn at 19 USDC.
USDC page	10 USDC (default)	Not crossed.
XLM warn	5 XLM (default)	Not crossed - spendable XLM is about 9997.5.
XLM page	2 XLM (default)	Evaluated against spendable XLM after ledger reserve, liabilities and sponsored reserves - not the gross balance.
Alert cooldown	15 minutes (default)	QA should be given a shortened QA-only value rather than waiting this out (part of B3).
Cap alert threshold	80 percent consumed (default)	Shares the same cooldown.

Alert identities TC-022 and TC-024 may encounter

Balance and rail conditions: wallet-usdc-warn, wallet-usdc-page, wallet-xlm-warn, wallet-xlm-page, payout-rate-spike, payout-volume-spike, payout-cap, repeated-payout-failures, reserve-refill-overdue.

Monitoring-source failures, which say the check itself could not run and must never be read as healthy:

wallet-monitoring-unconfigured, wallet-monitoring-unavailable, payout-monitoring-unavailable, reserve-monitoring-unconfigured, reserve-monitoring-unavailable, refill-timer-unavailable, health-monitor-unavailable, payout-persistence-unavailable.

Every alert delivery result is one of **sent**, **suppressed**, **sent-degraded**, **suppressed-degraded**, **failed**, or **disabled**. **On this build every result is disabled**, because no webhook is configured (B3). A paging alert is never dropped merely because the deduplication store is unavailable; a warning fails closed instead, so a warning storm cannot be amplified.

Scheduler

Nothing in this repository schedules any cron route. The four routes - wallet health, reserve refill, payout retry, payout reconcile - each require a bearer credential and are meant to be invoked by an external schedule, wallet health at least once per minute. On this build the credential is absent, so every request is rejected and no schedule exists (B2). TC-024 explicitly requires proof that a scheduler invokes monitoring rather than a manual call, and that requirement cannot be met until this is provisioned. There is no automated-test substitute for it: it is infrastructure, not code.

SECTION 10

Residual risks the QA record must carry

These are not caveats appended for completeness. Each one is a way the rail can still lose money or fail to warn, and the evidence package is incomplete without them. They are carried forward from #12, #73, the payments-lane evidence, and ADR-0001.

#	Residual risk	What it means, and what bounds it	Case
1	Co-signer isolation is simulated, at the server boundary only	The policy co-signer runs in its own service inside the same workspace, account, control plane, repository, CI pipeline and database instance as the application it is meant to be independent of. This satisfies server isolation and not account isolation. Bounded by: the level is machine-checked and refused on the public network, and an attacker holding the application container still cannot forge a payout. Accepted because the rail is pre-mainnet and the value at risk is zero.	TC-013 to TC-016, TC-025
2	The daily cap is a check, not a reservation	The cap reads the rolling total and authorises one payout against it. It takes no reservation, so concurrent payouts can each pass the cap and together exceed it . This is asserted as behaviour on purpose, so the trade-off cannot drift out of the documentation. Not at risk: no payout settles twice - every payout in an overshoot is a distinct payout the cap individually authorised.	TC-017, TC-019
3	Cap alerts are fire-and-forget	The alert is raised after the broadcast tuple is persisted - that ordering defect was fixed and is regression-guarded - but delivery itself is fire-and-forget at all three sites, so a store or webhook failure still loses that individual alert. Bounded by: the health monitor raises the same alert identity from the same ledger, which is the backstop. On this build neither is delivering (B2, B3).	TC-024, TC-026
4	The process-death window around envelope identity	The envelope hash that makes an ambiguous submit recoverable lives in memory for the duration of the call. If the process dies between submitting and resolving, that identity is lost and the job requeues without it - the one remaining path to a double settlement. The retry claim now holds a lease and refreshes it while a broadcast is in flight, which narrows the window. This is a lease, not a fence: refreshes are best-effort, a stalled event loop delays them, and process death is the case it cannot help with at all. Closing it needs the hash persisted before submit - a payout state-machine change.	TC-010, TC-011
5	The submit mutex is process-local	It serialises payouts inside one Node process and nothing beyond it, so the deployment must run exactly one payout submitter . A second instance costs throughput and retries rather than correctness - the loser gets a stale-sequence error, rebuilds once, then requeues - but scaling this path horizontally needs a distributed lock spanning the whole load, build, co-sign and submit cycle.	TC-009, TC-019
6	The retry lease is conservative and is not a fencing token	A last-retried timestamp cannot distinguish an in-flight retry from a recently completed failed one, so the lease errs toward refusing. Every claimant honours it from one shared definition: the cron stands down on a live lease, and the admin retry route refuses with a 409 rather than resetting a lease it does not own. The cost is a delayed legitimate retry, not an unsafe one.	TC-008, TC-012

Two limits outside the payments-lane issue's scope, and load-bearing anyway

Risks 4 and 5 above are the two. Neither is new and neither is closed by anything in Deliverable 1. Both are recorded here because the final QA record must state what was tested **without overstating the guarantees**. A QA record that omits them is not a stronger record; it is an inaccurate one.

The one thing that would be a blocking defect

Overshooting the daily cap under concurrency is expected and documented. **A duplicate or unreconciled settlement is not.** If any case produces two on-chain payments for one payout reference, or leaves a payout whose on-chain fate is unknown at the end of the run, that blocks approval regardless of every other result - and it must be reconciled before anything is retried.

SECTION 11

Safe evidence and redaction rules

The standard this document holds itself to, and the standard the executed evidence package must meet before #80 can be approved.

Never record	Record instead
Any Stellar secret seed - the 56-character values beginning with S	The corresponding public key beginning with G. Every verification in this suite is possible from public keys alone.
The HMAC shared secret, or any signed request header value	The HTTP status code and the redacted refusal reason. A 401 is the evidence; the header is not.
Database URLs, roles with passwords, or any connection string	The permission-probe outcome: which statement succeeded, which failed, and the permission-denied error class.
Webhook URLs	The alert identity, its severity, the delivery result word, and the timestamp.
Bearer tokens or cron credentials	That the call was authenticated, plus the response body with any secret field removed.
Admin or labeler passwords	The role used and the view reached.
A signed pending refill XDR	The transaction hash and the explorer link once it has been submitted, and the amount in units.
Raw environment output of any kind	The single variable name and whether it is set, absent, or invalid. Never run a command that prints the rest of the environment alongside it.
Raw error stacks or full log dumps from the payout path	The error class, the Horizon result code, the payout reference, and the transaction hash.

What is safe to publish, and is published here

- **Public keys beginning with G.** They are addresses. Every threshold, signer, weight and balance check in this suite reads them from a public explorer.
- **Testnet transaction hashes.** They are the evidence. All of them resolve on public testnet explorers.
- **Cap values and threshold numbers.** Not secrets - but read them back with a command that prints only the variable you asked for.
- **The isolation level, the network, the SHA, and the deployment timestamp.**

Two operational cautions from preparing this document

Filters that are too broad leak. A legacy private-key value was briefly exposed in operator terminal output during this review because a safe-variable filter matched more than intended. Read one named variable at a time, and treat any output that scrolled past a secret as compromised material - the fix is rotation, not deletion of the scrollbar.

Testnet keys are valueless; the credentials around them are not. A Stellar testnet seed controls no real funds. Database roles, admin passwords, cron credentials and webhook URLs are real regardless of which Stellar network they sit next to, and they are redacted on that basis.

SECTION 12

Handoff checklist and the QA:PASSED workflow

What has to be true before QA starts, and exactly what approval looks like when they finish.

Developer handoff checklist

#	Item	State
1	One stable testnet environment and build URL published	Done
2	Full commit SHA of the deployed build published	Done
3	Public on-chain evidence for payout account, signers, reserve, issuer, recipient and transactions published	Done
4	Live-versus-historical evidence reconciled and the runbooks corrected	Ships with this guide - verify at merge (B1)
5	Case-by-case execution mapping using the DOCX IDs published	Done - section 6
6	Residual-risk register published so QA need not reconstruct it from pull-request history	Done - section 10
7	Redaction standard published	Done - section 11
8	Cron credential provisioned and an external schedule invoking wallet health	Open - B2
9	QA alert channel configured and one delivery plus one suppression observed	Open - B3
10	Fault-injection controls for ambiguous submit and post-broadcast persistence failure, or #80's formal acceptance of the automated evidence path	Open - B4
11	Cold-reserve policy values configured on the web service	Open - B5
12	Redacted read-only-role permission probe captured, and the generic database URL removed from the cosigner service	Open - B6
13	One payout settled through the deployed remote co-signer	Open - B7
14	QA admin account issued out of band	Open - B8
15	Fixture and reset procedure published and rehearsed once	Open - B9
16	Shortened QA-only alert cooldown provided	Open - with B3
17	A CI run exists at the deployed SHA, or #80 accepts the pull-request head run and the merge chain	Open - B10
18	All QA-facing material reviewed for seeds, keys, HMAC values, database credentials, tokens and raw logs	Done for this document - re-run at each handoff

Items 1 to 7 and 18 are complete. Items 8 to 17 are the ten open blockers. **#13 is not complete and must not be marked complete until they are actually resolved**, not merely documented.

The simplified QA workflow

Approval is one label, and nothing else

QA applies the QA: PASSED label to issue #80 when every pass condition there is met. That label is the sole QA approval signal.

The following are explicitly **not required** of QA, and any instruction to the contrary in an older document is obsolete: a QA PASSED (full-sha) verdict comment; a Claude review link; closing this issue or any implementation issue; updating Epic #4; or any pull-request administration.

If any pass condition fails, QA does **not** apply the label. Record the failed and blocked case IDs and the linked defect issues in the test document and on #80. A blocked case is not a pass.

Who owns what

Owner	Issue	Responsibility
Engineering	#13	Environment, frozen build, evidence, fixtures, access, fault-injection mechanisms, this document, and the ten blockers. Notifies #80 if the deployed SHA changes.
QA	#80	Revising the test document, executing every applicable case against one SHA, recording results and redacted evidence, and applying the label.

Revisions the test document still needs (QA-owned)

These are the targeted tweaks section 1 refers to. None requires restructuring the suite.

- **Remove the obsolete gate framing.** The heading "#13 Final manual QA gate" and the "Final Manual QA Gate #13" subtitle now misdescribe the split: #13 is developer readiness and #80 owns execution.
- **Replace the verdict mechanic.** Delete "Record QA PASSED (full-sha)" from D1-TC-028 and the Final Execution Summary's verdict field, and replace it with applying the QA: PASSED label to #80. Keep the full SHA as a recorded field - the pass still binds to the exact commit.
- **Replace conditional phrasing with mechanisms.** "If supported", "where applicable", "controlled ability" and "according to the D1 scenario" each become a concrete step from section 6 or 7 of this guide, or an explicit statement that the case is blocked.
- **Fix the concurrency count.** D1-TC-009 and D1-TC-019 should name **twelve** concurrent payouts and the pass calculation: twelve distinct sequence numbers, zero duplicate ledger rows, zero duplicate on-chain transfers.
- **Add the residual-risk register** from section 10 as a required part of the final record, so the suite cannot pass while overstating the guarantees.
- **Add a requirement reference per case** so every acceptance criterion in #5 to #12 is traceable to at least one case ID.
- **Document usability.** Repeat the case ID and title on any continuation page, correct the clipped even-page header, and add a document version, revision date, QA owner, and the evidence-package location.

SECTION 13

Public and reference links

Everything cited above, in one place. All Stellar links are testnet.

Environment

What	Where
Application	https://centient.work
Railway public domain	https://web-production-05914d.up.railway.app
Public wallet health	https://centient.work/api/health/wallet
CI run carrying the green payments lane (pull-request head f0b3dd0625de, not the deployed merge commit - see B10)	https://github.com/webnxt-2030/Centient/actions/runs/34217894219

Issues and epic

Issue	Title	Link
#4	Epic - Deliverable 1	webnxt-2030/Centient/issues/4
#5	Multisig payout account	webnxt-2030/Centient/issues/5
#6	Testnet payout spike	webnxt-2030/Centient/issues/6
#7	Payout service	webnxt-2030/Centient/issues/7
#8	Independent policy co-signer	webnxt-2030/Centient/issues/8
#9	Daily payout cap	webnxt-2030/Centient/issues/9
#10	Hot/cold reserve	webnxt-2030/Centient/issues/10
#11	Wallet health	webnxt-2030/Centient/issues/11
#12	Payments-lane proof	webnxt-2030/Centient/issues/12
#13	Prepare Deliverable 1 for unrestricted QA execution	webnxt-2030/Centient/issues/13
#73	Deliverable 1 follow-up fixes and evidence	webnxt-2030/Centient/issues/73
#80	Deliverable 1 QA execution and test-case refinement	webnxt-2030/Centient/issues/80

On-chain evidence

What	Explorer link
Live payout account	account/GCP34RITQIVSLHS5T4XZRENIBUS3T7FHL3VSR24GK7HPMHGAAKWK4BU0
Live set-options transaction	tx/e966c0a5c27cbe0253f2812d158b38f8e91513de48254923f9ecb6f4c19630fd
Live two-signature payout	tx/5083dd72a16bfa749c6b302c293e931939acd70cc52f63586443921d60698206
Cold reserve account	account/GDPGRS4P6UZZK23CKKELGLJAYTCAWPV4C7TH6Q322SF735A5H6U5XK5G
USDC issuer	account/6BBD471F6LWK7P7MDEVSCWR7DPUWV3NY3DTQEVL4NAT4AQH3ZLLFLA5
Sponsored zero-XLM recipient	account/GA4XSHGIOVGWI6ZGDIRFG3R0FOHXRDVHH5YKGFKC5C2LH0T6EB5WCQXI

Repository documents

Document	Covers
docs/stellar-multisig-runbook.md	#5 signer set, thresholds, set-options procedure, key custody
docs/stellar-multisig-payout-spike.md	#6 sponsorship, fee-bump and zero-XLM recipient mechanics
docs/stellar-multisig-payout-service.md	#7 settlement path, sequence safety, failure modes
docs/stellar-cold-reserve-runbook.md	#10 refill ceremony, alert identities, Redis failure policy
docs/stellar-daily-payout-cap-runbook.md	#9 both cap gates, window divergence, change procedure
docs/stellar-payout-failure-runbook.md	failure-mode table and reconciliation procedure
docs/cosigner-deployment.md	#8 topology, read-only role and its permission probes
docs/payments-lane-evidence.md	#12 guard, definition of done, residual risks
docs/adr/0001-simulated-cosigner-isolation.md	the isolation decision and its same-project amendment
.env.local.example	every configuration variable with its meaning

External references

Reference	Link
Stellar testnet Horizon	https://horizon-testnet.stellar.org
Friendbot (testnet XLM funding)	https://friendbot.stellar.org
Circle testnet USDC faucet	https://faucet.circle.com
Stellar fee-bump transactions	https://developers.stellar.org/docs/learn/encyclopedia/transactions-specialized/fee-bump-transactions
Stellar sponsored reserves	https://developers.stellar.org/docs/learn/encyclopedia/sponsored-reserves

End of the Deliverable 1 QA Readiness Guide. Developer-owned handoff for issue #13, prepared 2026-09-08 against build fa36cf4663fecea6137dcd4e73b136baa49d34f0. Stellar Testnet only. QA execution, the revised test document, and the QA:PASSED approval label belong to issue #80.